

PSI – Zadanie 1.3

23.11.2025

Jakub Kwaśniak

Michał Pędziwiatr

Kacper Górski

Zadanie 1.3 (Wariant B)

Opis rozwiązania

Rozwiązanie z zadania 1.1 + 1.2 rozszerzone zostało o mechanizm **ABP (Alternating Bit Protocol)**, będący systemem typu „wyślij i czekaj na potwierdzenie”. Zgodnie z treścią wariantu B, do komunikacji wykorzystano protokół **IPv6: AF_INET6** w implementacji C klienta oraz **socket.AF_INET6** w przypadku serwera Python.

Po aktualizacji do każdego wysłanego pakietu, zarówno ze strony klienta jak i serwera, dodawany jest jednobajtowy nagłówek, zawierający numer sekwencyjny: 0 lub 1. Struktura datagramu z wcześniejszych zadań znajduje się bezpośrednio po nim.

Klient (**client_ipv6.c**) po nadaniu pakietu oczekuje na potwierdzenie odbioru sygnałem ACK, następnie porównuje jego numer sekwencyjny (**recv_seq**) z tym wysłanym przez niego w nagłówku pakietu (**seq_bit**). Dopiero gdy te wartości są identyczne, uznaje wysłany pakiet za odebrany. W przeciwnym wypadku, po upływie czasu oczekiwania (2 sekund) zdefiniowanego za pomocą **SO_RCVTIMEO**, ponawia wysłanie ostatniego datagramu.

Serwer (**udp_server_ipv6.py**) przechowuje dane ostatniego połączenia z klientem obejmujące: jego adres, ostatni odebrany numer sekwencyjny oraz ostatnią wysłaną odpowiedź. Po odebraniu datagramu, serwer sprawdza jego pierwszy bajt. Jeśli jest on identyczny z poprzednim numerem sekwencyjnym, serwer traktuje to jako duplikat. Skutkuje to zignorowaniem otrzymanej wiadomości oraz ponownym wysłaniem ostatniej, zbuforowanej odpowiedzi.

Wyniki testów

Zapis całości testów odtworzyć można dzięki zapisom sesji terminali równolegle uruchomionego skryptu klienta i serwera: **demo_client.cast** oraz **demo_server.cast**.

W celu symulacji warunków o znaczącej wadliwości łącza skorzystano z polecenia wprowadzającego opóźnienie oraz losową utratę 50% pakietów:

```
mpedziwi@bigubu:~$ docker exec z53_udp_abp_server_py tc qdisc add dev eth0 root netem delay 1000ms 500ms loss 50%
```

Klient poprawnie identyfikuje brak potwierdzenia odbioru w danym oknie czasowym. Komunikat „Timeout, retrying...” sygnalizuje ponowne przesłanie pakietu, powtarzane aż do skutku (otrzymania ACK od serwera):

```
mpedziwi@bigubu:~$ docker start -a z53_udp_abp_client_c

[CLIENT] Processing message 1 (Seq: 0)...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] Timeout, retrying...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] ACK received for Seq 0
Decoded datagram (2 pairs):
- 'status': 'OK'
- 'dg_size': '83'

[CLIENT] Processing message 2 (Seq: 1)...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] Timeout, retrying...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] ACK received for Seq 1
Decoded datagram (2 pairs):
- 'status': 'OK'
- 'dg_size': '83'

[CLIENT] Processing message 3 (Seq: 0)...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] Timeout, retrying...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] Timeout, retrying...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] Timeout, retrying...
[CLIENT] Packet sent, waiting for ACK...
[CLIENT] ACK received for Seq 0
Decoded datagram (2 pairs):
- 'status': 'OK'
- 'dg_size': '43'
```

Serwer, w sytuacji gdy klient ponownie przesyła mu wiadomość o tym samym numerze sekwencyjnym, wykrywa duplikat wiadomości i ignoruje jej zawartość („[SERVER] seq duplicate: 0”):

```
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] Decoded: {'name': 'Kacper', 'task': 'UDP-IPv6'}
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] seq duplicate: 0
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] Decoded: {'city': 'Warsaw', 'value': '2137'}
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] seq duplicate: 1
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] Decoded: {'hello': 'world'}
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] seq duplicate: 0
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] seq duplicate: 0
z53_udp_abp_server_py | [SERVER] Received datagram from ('fd00:1032:ac21:53::3', 42068, 0, 0)
z53_udp_abp_server_py | [SERVER] seq duplicate: 0
z53_udp_abp_client_c exited with code 0
```

Wnioski

Zaimplementowanie Alternating-Bit-Protocol zapewnia niezawodność transmisji w środowisku IPv6. Poprawia obsługę utraty pakietów poprzez ich retransmisję oraz eliminuje duplikaty po stronie serwera. Mechanizm ten wymaga jednak oczekiwania na potwierdzenie (sygnał ACK) każdego pakietu przed wysłaniem następnego, co może spowolnić i utrudnić przesyłanie danych.